



SOKOSMART: A MACHINE LEARNING
SYSTEM FOR DEMAND FORECASTING
AND INVENTORY MANAGEMENT IN
FASHION MICRO-RETAIL

By: Keziah Ndanu Masai

Admission: 24ZAD109796

A Project Proposal Submitted in Partial Fulfilment of the Requirements for the Degree
of Computer Science, School of Computing Sciences Riara University Nairobi,
Kenya.

JULY 20, 2026
RIARA UNIVERSITY
Course Code RBCS 7410

DECLARATION

I declare that this or any other University has not previously submitted this work for the awarding of the course marks. I also declare that I will only present this project once the supervisor has approved and signed the document.

To the best of my knowledge and belief, this work contains no material previously published or written by another person except where due reference is made.

(add student name)

(add signature)

(add date)

APPROVAL

This project proposal/ final project / dissertation of student was conducted under our supervision and is submitted with our approval as university supervisors.

Supervisor Name:

Signature:

Date:

DEDICATION

To my mother and my aunt,

I have watched you wake up while it was still dark outside. I have seen you come home with swollen feet, count the day's money, and still find energy to ask if we had eaten. You never called it resilience. You just called it a typical "Tuesday".

If this project works and I am praying it does I hope it buys you one extra hour of sleep. I hope it means you do not have to stand in the sun arguing with suppliers who refuse to pay what they owe. I hope it makes the math easier, not just for you, but for every woman who trades the way you do.

This is not just research. It is a thank you I could not afford to give you any other way.

With love and deep respect.

ACKNOWLEDGEMENT

Mother, thank you for everything, but especially for dragging me to the markets with you. I hated the 5 a.m. wake-ups. I hated the dust at Gikomba. But walking beside you while you argued with vendors and counted change in your palm taught me things no lecturer ever could.

Auntie, thank you for letting me into your business. I know I asked too many questions while you were trying to manage stock and handle customers. You still explained every decision, even the bad ones. That patience shaped this project more than any textbook.

To the vendors at Gikomba, Eastleigh, Toi, and in the CBD: you let me stand there and watch. You did not know I was writing things down. You were just trying to sell, and that ordinary effort is what I needed to see.

To my supervisors at Riara: thank you for the feedback, and for not letting me submit lazy work.

Finally, I used Claude during this research to help organise my arguments and check my writing. It was a useful tool, but the problems I am trying to solve came from the markets, not from a chatbot.

ABSTRACT

Fashion vendors in informal markets often manage their stock using notebooks, their previous sales experience, and/or personal judgement. Although these methods can work, they make it difficult for vendors to know when a product is likely to run out or when demand may increase. Platforms such as Shopify and QuickBooks already provide inventory management features, but they may not be practical for vendors who have limited access to computers, reliable internet, or technical training, and are small-scale with insufficient funding.

SokoSmart is my answer to that gap: a demand forecasting and inventory management system vendors can run entirely through WhatsApp. A vendor messages the SokoSmart number, Twilio hands that message to a Flask backend, and the system replies with whatever she asked for a demand forecast, current stock status, or a reorder alert, without her ever touching a dashboard.

Under the hood, I test ARIMA, XGBoost, and LSTM against each vendor's own sales history, compare them on RMSE, MAE, and MAPE, and deploy whichever performs best. Those forecasts then feed directly into the reorder-point logic behind the alerts.

Requirements come from a mix of direct observation at the target markets and a review of the relevant literature. Development follows Agile Scrum, working through sprints for system design, forecasting, inventory optimisation, chatbot integration, testing, and deployment, with the finished system hosted on Render or Railway. All a vendor needs is a smartphone, WhatsApp, and a data connection no technical background required.

Contents

DECLARATION.....	ii
ABSTRACT	v
LIST OF ABBREVIATIONS.....	viii
LIST OF FIGURES	ix
LIST OF TABLES.....	x
CHAPTER ONE	1
1.1. Problem Statement	1
1.2. Objectives of the Study	2
1.3. Justification.....	2
1.4. Scope.....	3
CHAPTER TWO	4
2.1. Introduction.....	4
2.2. Inventory Management in Small Scale Retail.....	4
2.3. Demand Forecasting Using Machine Learning.....	6
2.4. Inventory Optimization Techniques	9
2.5. Conversational AI and WhatsApp-Based System Deployment	10
2.6. Conclusion	11
CHAPTER THREE	12
3.0. Introduction.....	12
3.1. System Development Methodology	12
3.2 Requirements Gathering	14
Structured Observation.....	14
3.3 System Design.....	21
3.4 Technical Requirements.....	30
3.5 Project Budget.....	33
3.6 PROJECT TIMELINE (GANTT CHART).....	35
References.....	37
APPENDIX A: PLAGARISM REPORT	40
APPENDIX B: STRUCTURED OBSERVATION NOTES.....	45

DEFINITION OF TERMS

ARIMA (Autoregressive Integrated Moving Average) – A statistical model used in SokoSmart to predict future sales from past sales data.

Dynamic Reorder Point (ROP) – A stock level that changes according to expected demand and is used to decide when to reorder.

EOQ (Economic Order Quantity) – A method used to estimate an appropriate quantity of stock to order while considering ordering and storage costs.

Flask – A Python framework used to build the backend of SokoSmart.

LSTM (Long Short-Term Memory) – A neural network model used to identify patterns in sales data over time.

Machine Learning – The use of data to identify patterns and make predictions. SokoSmart uses it for demand forecasting.

MAE (Mean Absolute Error) – A measure of the average difference between predicted and actual sales values.

MAPE (Mean Absolute Percentage Error) – A measure of forecasting error expressed as a percentage.

RMSE (Root Mean Square Error) – A forecasting error measure that gives more weight to larger prediction errors.

Safety Stock – Extra stock kept to reduce the chance of running out of a product.

Scrum – The Agile framework used to organise the development of SokoSmart into sprints.

SokoSmart – The system developed in this project to forecast product demand and provide inventory alerts to fashion vendors through WhatsApp.

Stockout – A situation where a product is unavailable when a customer wants to buy it.

Twilio WhatsApp Business API – The service used to connect SokoSmart with WhatsApp.

Webhook – A method used to send incoming WhatsApp messages from Twilio to the SokoSmart backend.

XGBoost (Extreme Gradient Boosting) – A machine learning model used to predict sales by combining several decision trees.

LIST OF ABBREVIATIONS

Abbreviation Full Term

API	Application Programming Interface
ARIMA	Autoregressive Integrated Moving Average
CAK	Communications Authority of Kenya
CBD	Central Business District
DFD	Data Flow Diagram
EAT	East Africa Time
EOQ	Economic Order Quantity
ERP	Enterprise Resource Planning
ERD	Entity Relationship Diagram
FR	Functional Requirement
GDP	Gross Domestic Product
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IHL	Inventory Holding Loss
KES	Kenyan Shillings
KNBS	Kenya National Bureau of Statistics
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
MSME	Micro, Small and Medium Enterprise
NFR	Non-Functional Requirement
ORM	Object Relational Mapper
POS	Point of Sale
RAD	Rapid Application Development
RMSE	Root Mean Square Error
ROP	Reorder Point
SDK	Software Development Kit
SHA	Secure Hash Algorithm
SME	Small and Medium Enterprise
SQL	Structured Query Language
TLS	Transport Layer Security
XGBoost	Extreme Gradient Boosting

LIST OF FIGURES

Figure 1 Agile Scrum Development Process for SokoSmart.....	14
Figure 2 use case diagram.....	22
Figure 3 Block diagram.....	23
Figure 4 Data Flow Diagram.....	24
Figure 5 Entity relationship diagram	25

LIST OF TABLES

Table 1 Characteristics of all the three models	8
Table 2 organization of the sprints	12
Table 3 identified non-functional requirements	16
Table 4 identified functional requirements	18
Table 5 Data Dictionary	26
Table 6 Hardware Requirements	30
Table 7 programming languages	30
Table 8 Frameworks and Libraries	31
Table 9 Development Tools	31
Table 10 Cloud Deployment Tools	32
Table 11 Hardware requirements	32
Table 12 Software Requirements	33
Table 13 Budget	33
Table 14 Gantt Chart SokoSmart Development Schedule	35

CHAPTER ONE

1.1. Problem Statement

Small-scale fashion vendors in Gikomba, Eastleigh, Toi Market, and the Nairobi CBD know their demand patterns exist they just can't predict them. Demand climbs before schools reopen, spikes around end-of-month paydays, and drops off once a trend fades, and most vendors are still tracking all of this with a notebook, phone notes, or plain memory.

What they'd ideally want is to use their own sales history to plan ahead, but almost nothing on the market makes that practical for a small informal operation. Shopify and QuickBooks both offer inventory tools, but they're built around computers, reliable internet, and comfort with digital dashboards, none of which describes most of Nairobi's informal retail scene.

Without something affordable, vendors are left guessing how much to buy, and guessing wrong in either direction is costly: overstocking ties up capital they don't have to spare, understocking sends customers straight to the next stall.

Without an affordable and accessible forecasting tool, vendors have to rely mainly on their own judgement when deciding how much stock to purchase. Globally, inventory mismanagement is estimated to cost retailers approximately \$1.75 trillion annually (IHL Group, 2023).

That's the specific gap SokoSmart is built for: vendors sitting on genuinely useful sales history with no practical way to turn it into a forecast. Machine learning has already shown it can sharpen inventory decisions in retail (Fildes et al., 2022), but almost all of that work assumes structured digital records and users comfortable with more complex software, neither of which describes a Gikomba stallholder.

SokoSmart closes that gap by putting a machine-learning forecasting and inventory system behind a WhatsApp chatbot. A vendor's message goes to the SokoSmart number, gets picked up by a Flask backend through Twilio, and comes back as a plain-language forecast, stock status, or low-stock alert nothing else required on her end. ARIMA, XGBoost, and LSTM are all tested against her actual sales history, the best performer (by RMSE, MAE, and MAPE) gets selected, and its output drives the reorder-point calculations behind the alerts.

The system is specifically intended for small-scale fashion vendors in Gikomba, Eastleigh, Toi Market, and the Nairobi CBD. Vendors will access it through WhatsApp rather than installing a separate application or using a computer-based dashboard. The project is limited to demand forecasting, reorder-point calculation, and WhatsApp-based inventory alerts. It does not cover payment processing, supplier

management, multi-language support, or full business accounting.

1.2. Objectives of the Study

1.2.1. General Objective

To develop a machine learning-based demand forecasting and inventory management system for small-scale fashion vendors in Nairobi, Kenya, using a WhatsApp chatbot interface.

1.2.2. Specific Objectives

1. To identify the inventory management challenges and information needs of small-scale fashion vendors in selected Nairobi markets through literature review and market observation.
2. To develop SokoSmart using ARIMA, XGBoost, and LSTM forecasting models, reorder-point calculations, inventory alerts, and a WhatsApp chatbot interface using Twilio and Flask.
3. To evaluate the forecasting performance of the implemented models using RMSE, MAE, and MAPE.
4. To assess the usability of the SokoSmart WhatsApp chatbot through user testing.

1.3. Justification

Kenya's informal fashion retail sector contributes meaningfully to the economy, yet most of it still runs on notebooks, memory, and gut instinct rather than any structured forecasting. That gap isn't free: inventory mismanagement costs retailers roughly \$1.75 trillion a year worldwide (IHL Group, 2023), and micro-enterprises typically feel it harder, since they have less working capital to absorb a bad stocking decision.

The frustrating part is that the fix already exists elsewhere machine learning forecasting has been shown to cut inventory-related costs by 20–50% in comparable retail settings (Fildes et al., 2022) it just hasn't reached Kenyan fashion micro-vendors, largely because platforms like Shopify and QuickBooks are priced and built for businesses with more infrastructure than a market stall has. SokoSmart is my attempt to close that distance by delivering the same forecasting power through an interface vendors are already using every day: WhatsApp. Since most of them communicate with suppliers and customers on it already, adopting SokoSmart shouldn't require new hardware, new software, or any real training (Communications Authority of Kenya, 2024).

1.4. Scope

SokoSmart does not cover everything, but it covers the following. In terms of functionality, the system covers **demand forecasting and automated inventory reorder alerts only**. Payment processing, supplier management, POS hardware, and financial reporting are excluded from the project.

For data, training and evaluation it will use **publicly available fashion retail sales datasets augmented with locally relevant parameters**. The collection of live vendor transaction data is not within the scope of the project.

Regarding the interface, the WhatsApp chatbot supports **text-based interactions only**. Voice input, image-based stock capture, and multi-language support beyond English are excluded from the current system.

From a technology perspective, the system uses **Python open-source libraries**, including scikit-learn, TensorFlow, and statsmodels, together with a Flask backend and the Twilio WhatsApp Business API. The system will be deployed using Render or Railway cloud hosting.

Geographically, the project focuses on **Nairobi County, Kenya**, specifically targeting fashion vendors operating in **Gikomba, Eastleigh, Toi Market, and the Nairobi CBD**. The intended users are individual fashion micro-vendors who operate their businesses through smartphones with standard mobile data. No prior technical background is required to use the system.

Finally, the success of the system will be measured using **RMSE, MAE, and MAPE** to evaluate forecasting accuracy. The usability of the WhatsApp interface will also be assessed through structured testing with representative users.

CHAPTER TWO

2.1. Introduction

Building SokoSmart meant reading across several fields that don't usually sit in the same paper: inventory management in informal retail, forecasting methods old and new, stock optimisation techniques, and conversational AI delivered over WhatsApp. This chapter works through what that reading turned up, starting with how small and micro-retailers, especially in developing economies, currently handle stock, then moving into the statistical and machine-learning models behind demand forecasting (with a closer look at the three SokoSmart actually uses: ARIMA, XGBoost, and LSTM), before covering reorder-point models and automated alerts, and finally chatbot-based delivery through WhatsApp specifically in the Kenyan context. By the end, the gaps in what's already out there should explain why SokoSmart looks the way it does.

2.2. Inventory Management in Small Scale Retail

Every retail business has to manage inventory in some form — ordering, storing, tracking, and controlling stock so demand gets met without costs spiralling. Large retailers do this through ERP systems and dedicated supply chain teams. Small and micro-retailers, especially in informal economies, mostly have neither: they run on manual records, personal experience, and intuition, which makes their stocking reactive rather than predictive.

That reactive posture leaves small retailers exposed on both sides. Buy too much and working capital gets tied up in stock that, in fashion retail, risks going out of trend before it sells (Ren et al., 2020). Buy too little and shortages cost sales, frustrate customers, and send them to a competitor down the row. Kharfan et al. (2020) found fashion retail particularly punishing here, since demand shifts quickly with season and taste miss the restocking window and an entire season's sales can be gone. For vendors already working with limited capital and little access to credit, either mistake carries real financial weight.

Traditional practices periodic manual counts, rule-of-thumb reorder decisions, experience built up over years of trading still get small retailers through day-to-day operations, but they fall short once demand shifts often, product ranges grow, and seasonal buying becomes the norm. Ferreira et al. (2016) showed what data-driven inventory management can do for an online retailer with rich transaction records; whether that holds

for micro-vendors working from thin, manual records is a big part of what this study is testing.

2.2.1. Inventory Management Challenges in Developing Economy Retail

Kenya's small fashion vendors face more than seasonal demand swings. Unreliable electricity, patchy internet, expensive mobile data, and the absence of formal point-of-sale systems all mean the transaction data that would normally support inventory decisions simply doesn't exist (Wanjiru & Ngugi, 2021). Wanjiru and Ngugi's study of Nairobi County SMEs found most retailers still tracked stock with notebooks or phone reminders and that businesses with more structured inventory practices tended to perform better financially, which is a useful signal even though their study observed existing practice rather than testing any digital intervention.

Nairobi's markets add their own layer of complexity on top of that. Vendors in Gikomba, Eastleigh, Toi Market, and the CBD serve customers whose buying patterns shift with salary cycles, school terms, and nearby competition (Omwenga et al., 2020). None of which shows up in forecasting models built on retail data from wealthier markets. Omwenga et al. identified these local demand patterns but stopped short of building or testing any tool vendors could actually use, which is part of the gap this project is trying to close.

Another challenge is the digital divide within Kenya's informal retail sector. Many existing inventory management systems assume that businesses have access to desktop computers, reliable broadband internet, or staff with technical skills. These assumptions do not reflect the reality faced by many small vendors. The Communications Authority of Kenya (2024) reports that smartphone ownership and WhatsApp use are widespread among informal traders in Nairobi, whereas laptop ownership and fixed broadband access remain limited. For this reason, SokoSmart is designed to operate entirely through smartphones using WhatsApp. However, the CAK report measures access to devices rather than how vendors use them for business activities. High WhatsApp adoption does not necessarily mean that vendors will automatically use a chatbot for inventory management, making user acceptance an important consideration for this study.

2.3. Demand Forecasting Using Machine Learning

As datasets grew and computing got cheaper, machine learning and deep learning started beating traditional statistics outright especially where demand is non-linear, shaped by several factors at once, and spread across a wide product range. Most of that evidence, though, comes from data-rich settings, so it's genuinely unclear how well these models hold up once historical records get thin..

Swaminathan et al. (2023), in a systematic review of demand forecasting methods for fashion products published in the *International Journal of Forecasting*, found that machine learning and deep learning models consistently outperformed classical statistical approaches across several fashion retail datasets. The performance gap became larger as product variety and demand volatility increased. The authors identified LSTM neural networks and gradient boosting methods as among the strongest-performing model families for fashion-specific forecasting tasks, noting that their ability to capture non-linear relationships and complex seasonal patterns gave them an advantage over ARIMA-type models in volatile retail environments. However, the review was based mainly on studies conducted in developed markets with strong data infrastructure, making it unclear whether the same advantage is maintained under data-scarce conditions.

Ren et al. (2020) found that folding in extra signals social media trends, weather, promotional calendars alongside historical sales pushed machine-learning forecasts well past what sales data alone could deliver. That's encouraging, but it's also a problem for SokoSmart specifically: micro-vendors don't have access to any of those enriched datasets. The system has to produce something useful working from sales history alone.

2.3.1. Comparative Analysis of Forecasting Models: ARIMA, XGBoost, and LSTM

SokoSmart uses three forecasting models ARIMA, XGBoost, and LSTM spanning traditional statistics through to modern deep learning. Lining them up against each other explains why each one made the cut and gives some grounding for interpreting the results later in this study.

ARIMA (Autoregressive Integrated Moving Average) is one of the most widely used statistical models for time series forecasting. It predicts future values by analysing patterns in historical sales data through autoregressive and moving average components. Because of its simplicity, interpretability, and relatively low computational requirements, ARIMA continues

to be used in many retail forecasting applications (Makridakis et al., 2018). Its main limitation is the assumption that relationships within the data are largely linear. In fashion retail, where demand is influenced by changing trends, promotions, and other external factors, this assumption can reduce forecasting accuracy. Within SokoSmart, ARIMA provides a baseline against which the machine learning models are evaluated. Its performance also depends on the availability of clean and consistent historical sales records, something that many vendors using manual record-keeping may not have.

Unlike ARIMA, **XGBoost (Extreme Gradient Boosting)** is a machine learning algorithm that combines multiple decision trees to improve prediction accuracy. Each new tree is built to reduce the errors made by the previous ones, allowing the model to learn complex relationships within structured data. Swaminathan et al. (2023) identified gradient boosting techniques among the strongest-performing approaches for fashion demand forecasting because they can use multiple variables at the same time. Loureiro et al. (2018) also demonstrated that advanced learning models can improve sales forecasting accuracy in fashion retail, suggesting that structured ensemble methods such as XGBoost are well suited to similar forecasting tasks. For SokoSmart, this flexibility is important because the model can incorporate factors such as the day of the week, month, promotional activities, and product category when generating forecasts.

LSTM (Long Short-Term Memory) networks represent the deep learning approach used in this project. As a specialised form of recurrent neural network, LSTM is designed to learn long-term patterns within sequential data, making it suitable for forecasting demand that changes over time. Giri et al. (2022) reported that LSTM models captured complex seasonal behaviour and irregular demand patterns more effectively than both ARIMA and traditional machine learning methods in fashion forecasting simulations. Similarly, Seyedan et al. (2023) found that deep learning models incorporating LSTM components produced greater reductions in inventory costs than non-deep learning alternatives in online retail environments. The trade-off is that LSTM requires more computational resources and larger volumes of historical data during training. Although this does not affect real-time

Dimension	ARIMA	XGBoost	LSTM
-----------	-------	---------	------

Model type	Statistical	Ensemble ML	Deep learning
Handles non-linearity	No	Yes	Yes
Multi-variable input	Limited	Yes	Yes
Captures long-term patterns	Partially	Partially	Yes
Interpretability	High	Moderate	Low
Data requirement	Low	Moderate	High
Computational cost	Low	Moderate	High
Fashion retail performance	Baseline	Strong	Strongest
Suitable for SokoSmart	As baseline	Primary candidate	Primary candidate

forecasting after deployment, it may reduce

performance when vendors have only limited historical sales records.

The table summarises the main characteristics of the three forecasting models, highlighting their strengths, limitations, and suitability for the operational requirements of SokoSmart.

Table 1 Characteristics of all the three models

Table 1 makes the trade-off obvious: no single model wins outright, which is exactly why SokoSmart doesn't commit to one in advance. Instead, all three get evaluated on RMSE,

MAE, and MAPE, and whichever comes out ahead gets deployed following the same evaluation approach Makridakis et al. (2018) recommend for retail forecasting.

2.4. Inventory Optimization Techniques

Forecasting tells a vendor what might sell; it does not tell her when to reorder or how much. That is where inventory optimisation comes in. It uses methods from operations research and ML to help with stock management. Two common methods are the Economic Order Quantity (EOQ) model and the Reorder Point (ROP) model. Using these methods together with demand forecasts can make inventory decisions more responsive to changes in demand (Seyedan et al., 2023).

The Economic Order Quantity (EOQ) model is used to find an order quantity by considering ordering costs and holding costs. One limitation of the model is that it assumes demand is constant and predictable. This may not work as well for small retail businesses where demand can change because of seasonal trends and unpredictable buying patterns. Baryannis et al. (2019) found that ML based forecasting generally performed better than static models when demand was highly uncertain. However, their review did not specifically look at micro-retail businesses.

Reorder Point (ROP) model is used to decide when a new order should be made. The traditional calculation uses average daily demand, lead time, and safety stock to reduce the chance of running out of stock. In a system that uses demand forecasts, the reorder point can be changed as the expected demand changes. SokoSmart uses this approach by updating the reorder threshold after each forecasting cycle. The low-stock alert can therefore be based on the current demand pattern instead of using one fixed stock level for every situation.

Hollupathirage (2025) studied a prototype digital twin system for retail inventory and sales management and found that AI-supported reorder point systems with real-time monitoring reduced stockouts compared with manual inventory tracking. The study also reported a reduction in excess inventory. The digital twin system used in the study is more advanced than the approach used in SokoSmart, so its results cannot be used to directly prove that SokoSmart will produce the same results. However, the findings are relevant to the idea of using forecasts to make reorder decisions instead of depending only on manual stock tracking. This is the approach used for the low-stock alert feature in SokoSmart.

2.5. Conversational AI and WhatsApp-Based System Deployment

SokoSmart runs entirely on WhatsApp, and that choice came directly out of what I saw at the four markets: every vendor had a smartphone, but almost none of them had an inventory app, spreadsheet, or POS system. WhatsApp, on the other hand, was open on nearly every screen I saw. Asking vendors to download and learn a separate app felt like exactly the kind of barrier that kills adoption before it starts.

Selamat and Windasari (2021) found something similar with chatbot adoption among small business owners: natural-language interaction lowered the technical barrier for people without formal software training. Their focus was customer service rather than stock control, but the logic carries over: if a vendor can simply read "this item will run out in three days" instead of navigating a dashboard, she can act on it immediately.

WhatsApp's position in Kenyan informal sectors makes this practical. The Communications Authority of Kenya (2024) backs this up too, reporting near-universal WhatsApp use among Nairobi's informal traders.. Jack & Suri (2014) traced how M-Pesa reached non-technical users by operating through the mobile phones they already owned; Aker & Mbiti (2010) noted that mobile platforms across sub-Saharan Africa succeeded precisely because they did not require users to acquire new devices or skills. SokoSmart follows that precedent by placing inventory forecasting inside an application that vendors already check repeatedly throughout the trading day.

This integration is intended to address the stock management problem directly. With manual systems, a vendor may only realise that an item is running out when a customer asks for it or when a physical stock count shows that it is no longer available. The WhatsApp chatbot changes this process by using the available stock information to generate forecasts and send alerts before the predicted depletion point. Since the warning appears in the same application the vendor already uses to communicate with suppliers, the vendor can act on it without having to switch to another system.

The Twilio WhatsApp Business API provides the messaging infrastructure, routing vendor inputs to the Flask backend and returning system responses. Development can proceed within Twilio's sandbox environment, avoiding the need for a verified

business account during the prototype phase. Deployment on Render or Railway supplies cloud hosting at low cost, though existing literature has not yet documented this particular combination of technologies for inventory management in informal retail settings.

2.5.1 Mobile First Systems in Kenyan Informal Retail

Kenya's informal economy has already shown that mobile-first platforms can work. M-Pesa proved this back in 2007 it spread to non-technical users who did not even own smartphones at the time, and later grew into smartphone-based business tools (Jack & Suri, 2014). Aker and Mbiti (2010) make a similar point: mobile phones across sub-Saharan Africa cut transaction costs and opened markets that small traders could not previously reach. The Communications Authority of Kenya (2024) shows this trend has held almost every informal trader in Nairobi now has a smartphone and uses WhatsApp. But here is the problem Wanjiru and Ngugi (2021) identify: having a phone has not changed how vendors manage stock. They still use notebooks and memory, even though the device in their pocket could do far more. That gap between owning the technology and using it for business operations is exactly where SokoSmart sits. M-Pesa handled money. WhatsApp handled messaging. Nobody has built a mobile-first tool for demand forecasting and inventory control aimed at these vendors. SokoSmart tries to fill that space.

2.6. Conclusion

Pulling this chapter together: the research on inventory management, forecasting, reorder optimisation, and chatbot accessibility is all out there and it all points the same direction. Machine learning beats traditional forecasting; dynamic reorder points beat static ones; chatbots lower the barrier to using digital tools at all. What's missing is somebody putting all four pieces together for vendors who don't have clean data, computers, or technical support which describes most of Nairobi's informal fashion vendors, not the reliable-data, well-resourced businesses most of this literature was actually tested on.

That's the gap SokoSmart sits in. Whether LSTM and gradient boosting hold up on the kind of small, patchy, manually-kept records these vendors actually have is still an open question going into this project one the evaluation in later chapters is meant to answer.

CHAPTER THREE

3.0. Introduction

This chapter covers how SokoSmart is being designed and built. I start with why I chose Agile Scrum as the development approach, then walk through how requirements were gathered and what they turned up. From there I present the system design the block diagram, data flow diagram, entity relationship diagram, and use case diagram followed by the hardware, software, and security requirements the build depends on. I close the chapter with the budget and the ten-week Gantt timeline.

3.1. System Development Methodology

SokoSmart doesn't lend itself to a single linear build. The forecasting models, the inventory optimisation logic, the Flask backend, and the WhatsApp chatbot all needed to be built and tested somewhat independently, and some of them were bound to change once testing exposed problems which is exactly why I went with Agile Scrum rather than a waterfall approach. Development happens in short cycles: build a piece, test it, review it, fix what's broken, move on. This suited the ML side of the project especially well, since ARIMA, XGBoost, and LSTM all had to be trained and compared on RMSE, MAE, and MAPE before I could settle on which one to deploy, and the chatbot itself needed real usability testing to see whether people could actually use it without help.

Selected Methodology: Agile Scrum

SokoSmart uses Agile with the Scrum framework.

Development is carried out in short cycles, where a part of the system is developed, tested, reviewed, and improved before moving to the next stage. This approach fits SokoSmart because the machine learning component requires experimentation. ARIMA, XGBoost, and LSTM need to be trained and compared using RMSE, MAE, and MAPE before the most suitable model can be selected. The WhatsApp chatbot also requires testing to determine whether the interaction is clear and practical for the intended users.

The development of SokoSmart was divided into four sprints, as shown below:

Table 2 organization of the sprints

Sprint	Focus	Duration
Sprint 1	Requirements gathering, system design, dataset sourcing	2 weeks
Sprint 2	Data preprocessing, ARIMA, XGBoost and LSTM model development	3 weeks

Sprint	Focus	Duration
Sprint 3	Inventory optimization module and Flask API development	2 weeks
Sprint 4	WhatsApp chatbot integration, testing, evaluation and deployment	3 weeks

Sprint 1 laid the groundwork; the project focused on understanding the inventory management problem, collecting the system requirements, designing the initial system structure, and finding the datasets needed for forecasting. These activities provided the starting point for the rest of the development work.

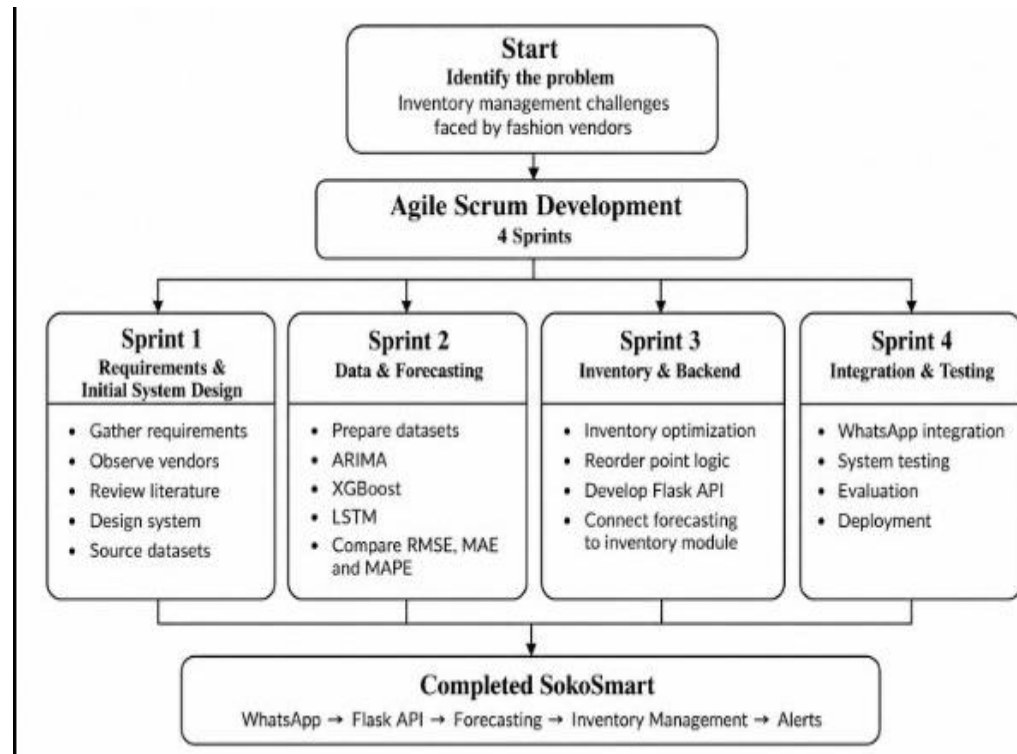
Sprint 2 moved into modelling: ARIMA, XGBoost, and LSTM were built, trained, and compared on RMSE, MAE, and MAPE to settle on the strongest performer. The results helped determine which forecasting model was most suitable for use in SokoSmart.

Sprint 3 wired the winning model into the inventory logic and Flask API. The forecasting results were linked to the inventory logic so that the system could identify when stock reached the reorder point. The Flask API handled the connection between the backend processes and the chatbot interface.

Sprint 4 tied everything to WhatsApp before finalizing. The different components were brought together at this stage, allowing a vendor to access the forecasting and inventory functions through WhatsApp.

Agile Scrum was chosen because it allowed SokoSmart to be developed in smaller stages instead of waiting until the whole system was finished before testing it. Problems found during model evaluation or chatbot testing could then be addressed while development was still in progress. This approach was useful for SokoSmart because the forecasting component and the user interface require different types of testing, but both still have to work together as one system.

Figure 1 Agile Scrum Development Process for SokoSmart



3.2 Requirements Gathering

3.2.1 Collection Method

Requirements were gathered using two main approaches: structured observation of vendors in the target market areas and a review of published literature.

Structured Observation

Direct observation was carried out across the four market areas: Gikomba, Eastleigh, Toi Market, and the Nairobi CBD. The purpose was to understand how small-scale fashion vendors manage their inventory in their day-to-day work. During the observation sessions, the researcher looked at the tools vendors used to keep track of stock, what caused them to restock, and some of the problems that could be seen when inventory was not managed well. These included piles of slow-moving items and empty displays when popular items had run out.

Several patterns were observed:

- **Stock tracking:** All observed vendors used manual methods such as handwritten notebooks, basic phone

notes, or memory. No vendor was observed using a digital inventory tool or spreadsheet.

- **Restocking triggers:** Restocking was mainly based on seeing that stock was running low or having enough cash to purchase more stock. Vendors were observed counting the items they had left or estimating stock levels by looking at the items on display rather than using a calculated threshold or forecast.
- **Overstock evidence:** Several stalls had items from previous seasons being sold at reduced prices. This suggested that earlier purchasing decisions had not accounted for changes in demand.
- **Stockout evidence:** Vendors were observed turning customers away or directing them to nearby stalls when particular sizes or colours were unavailable.
- **Technology use:** All observed vendors used smartphones. WhatsApp was visible on the screens for supplier communication and customer orders. However, no vendor was observed using an inventory management or POS application.

These observations provided firsthand evidence that the manual, intuition driven inventory practices documented by (Wanjiru & Ngugi, 2021) remain prevalent in the field, and confirmed that the operational context described in Chapter 1 is current and observable rather than purely theoretical.

Literature Review as Secondary Collection

The systematic literature review conducted in Chapter 2 provided secondary evidence that corroborated and extended the observation findings. Studies on inventory management challenges in developing economy retail (Omwenga et al., 2020; Wanjiru & Ngugi, 2021) confirmed the persistence of manual tracking methods and the impact of localised demand drivers. Machine learning forecasting literature (Giri et al., 2022; Makridakis et al., 2018; Swaminathan et al., 2023) established the technical benchmarks for model evaluation and selection. Conversational AI deployment studies (Selamat & Windasari, 2021) and the (Communications Authority of Kenya, 2024) supported the choice of WhatsApp as the delivery channel. Together, observation and literature review provided a complete evidence base for defining SokoSmart's requirements.

3.2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints that SokoSmart must satisfy they describe how the system performs rather than what it does (Pressman & Maxim, 2020). The following non-functional requirements were identified through the requirements gathering process:

Table 3 identified non-functional requirements

	Requirement	Description	Source
NFR1	Accessibility	The system must be operable by users with no prior technical training, delivered entirely through WhatsApp text interactions	Field observation + Literature
NFR2	Performance	Forecast queries submitted via WhatsApp must receive a response within 10 seconds under standard mobile data conditions	Literature
NFR3	Accuracy	The demand forecasting module must achieve a Mean Absolute Percentage Error (MAPE) of below 15% on test data	Literature
NFR4	Reliability	The system must maintain at least 95% uptime during operating hours (6am–10pm EAT)	Literature
NFR5	Scalability	The Flask backend must support at least 50 concurrent WhatsApp user sessions without performance degradation	Literature

	Requirement	Description	Source
NFR6	Affordability	The system must be deployable on cloud infrastructure costing under KES 2,000 per month. This threshold represents approximately 5–8% of the typical monthly mobile data and communication budget reported by Nairobi informal vendors (Communications Authority of Kenya, 2024), ensuring the operational cost does not impose a financial burden comparable to existing commercial platforms.	Field observation + Literature
NFR7	Security	Vendor sales data stored or processed by the system must not be shared with third parties and must be handled in compliance with Kenya's Data Protection Act 2019 (Republic of Kenya, 2019)	Legislation
NFR8	Maintainability	System components ML models, Flask API,	Literature

	Requirement	Description	Source
		WhatsApp integration must be modular and independently updatable without full system redeployment	
NFR9	Usability	First-time users must be able to complete a stock forecast query without external assistance within 3 minutes of first interaction	Field observation + Literature
NFR10	Portability	The system must function on any Android or iOS smartphone running WhatsApp version 2.21 or later	Field observation

3.2.3 Functional Requirements

Functional requirements define the specific behaviours, features, and functions that SokoSmart must perform (Sommerville, 2016). They describe what the system must do to satisfy the needs of its users.

The following functional requirements were derived from observation findings and literature review:

Table 4 identified functional requirements

	Requirement	Description	Source
FR1	Sales data input	The system must allow vendors to input historical sales data by product category and date through the	Field observation

	Requirement	Description	Source
		WhatsApp interface	
FR2	Demand forecasting	The system must generate product level demand forecasts for a user specified future period using ARIMA, XGBoost, and LSTM models	Literature
FR3	Model selection	The system must automatically select and deploy the best-performing forecasting model based on RMSE, MAE, and MAPE evaluation on the vendor's dataset	Literature
FR4	Inventory tracking	The system must allow vendors to log current stock levels by product category through the WhatsApp interface	Field observation
FR5	Reorder point calculation	The system must compute dynamic reorder points based on forecasted demand and user-defined lead time, updating automatically	Literature

	Requirement	Description	Source
		with each new forecast cycle	
FR6	Low-stock alerts	The system must send automated WhatsApp alerts to vendors when current stock levels fall at or below the computed reorder point for any tracked product	Field observation
FR7	Forecast query	The system must respond to a vendor's natural language WhatsApp query with a plain language forecast summary for the requested product and time period	Field observation
FR8	Stock status query	The system must respond to a vendor's WhatsApp query with the current stock level, reorder point, and alert status for any tracked product	Field observation
FR9	Data storage	The system must securely store vendor sales history and stock records for use in model training and	Literature

	Requirement	Description	Source
		reorder calculations	
FR10	Onboarding	The system must guide a first-time vendor through initial setup product registration, stock level entry, and forecast preference through a structured WhatsApp conversation flow	Field observation

3.3 System Design

The requirements from Section 3.2 need to become something concrete before anyone can build it. Four diagrams do that work here: the block diagram gives the high-level shape of the system, the data flow diagram traces how information moves through it, the entity relationship diagram lays out the data layer, and the use case diagram maps how the vendor and the system interact. Together they give enough of a blueprint to actually start building SokoSmart (Pressman & Maxim, 2020).

3.3.1 Use Case Diagram

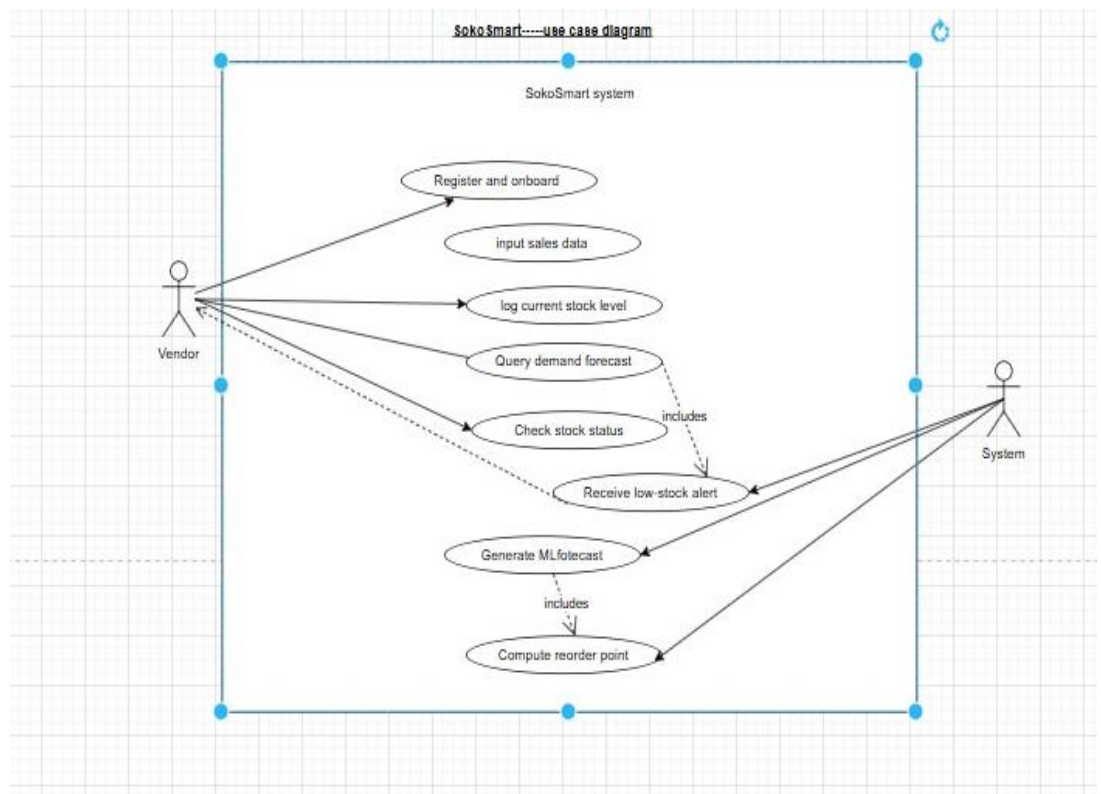


Figure 2 use case diagram

Figure 2 lays out two actors and eight use cases. The vendor drives five of them directly by messaging the system onboarding, logging sales data, logging stock, asking for a forecast, and checking stock status. The system runs the other three on its own: pushing low-stock alerts (shown with a dashed line, since it's the system reaching out rather than the vendor asking), generating ML forecasts, and computing reorder points. Querying a forecast automatically triggers a model run, and every model run recalculates the reorder threshold behind it that chain is what the include relationships in the diagram capture. Teal marks the vendor-facing use cases, coral the internal ML processes, and purple the hybrid alert that starts on the system side but lands with the vendor.

3.3.2 Block Diagram

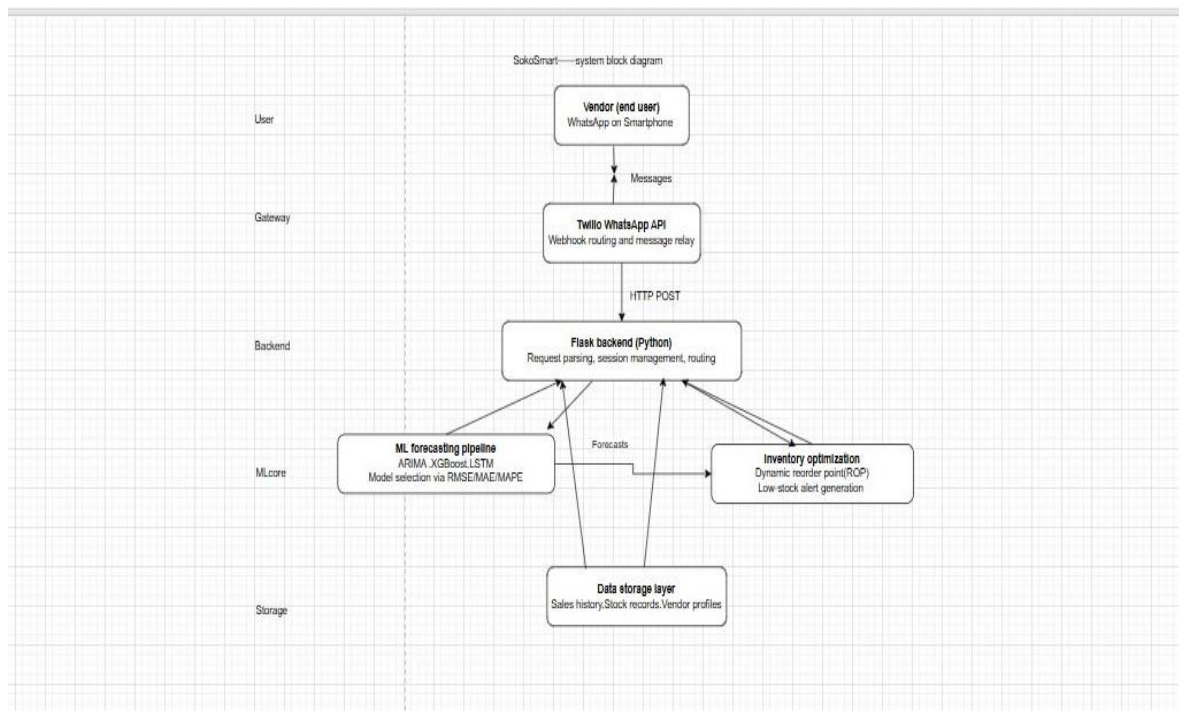


Figure 3 Block diagram

The block diagram (Figure 3) lays out SokoSmart's five layers. At the top sits the vendor layer the end user, interacting entirely through WhatsApp on a smartphone, with no other software or hardware required.

Below that is the gateway layer: the Twilio WhatsApp Business API, which receives incoming vendor messages, routes them to the backend over HTTP POST webhooks, and sends responses back the same way. The backend itself is a Flask application handling request parsing, session management, and routing between components.

The ML core does two things in parallel the forecasting pipeline (which trains the models and picks whichever performs best on RMSE, MAE, and MAPE) and the inventory optimisation engine (which turns those forecasts into reorder points and low-stock alerts). Underneath it all, the storage layer holds vendor sales history, stock records, and profiles, written to and read from continuously as vendors enter data and forecasts get regenerated.

One design decision worth flagging: if all three models fail to produce a usable output, or if a vendor simply hasn't logged 30 data points yet, the system falls back to a 30-day simple moving average rather than returning nothing. Models retrain automatically every Sunday at 2 AM EAT, or sooner if a vendor logs 50 new sales records first.

3.3.3 Data Flow Diagram

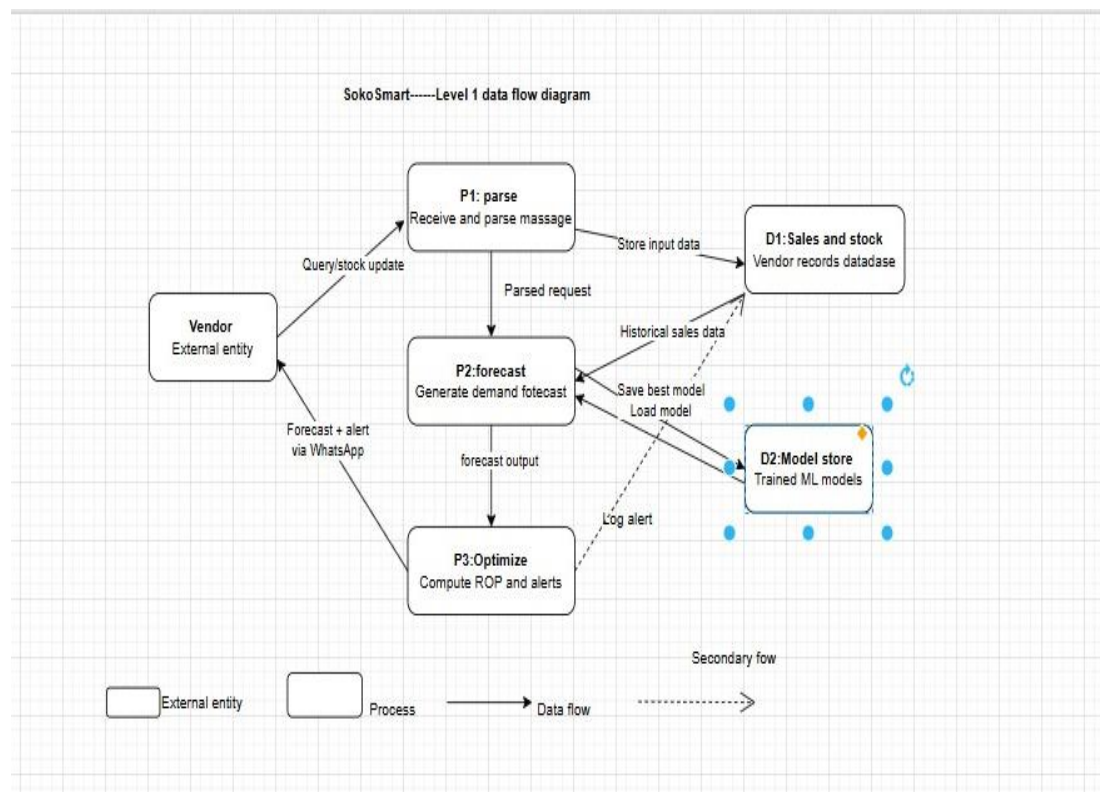


Figure 4 Data Flow Diagram

Figure 4 traces how a message actually moves through the system. The vendor is the only outside actor everything starts with her sending a stock update, forecast query, or status check over WhatsApp. Process 1 parses that message and writes anything new to the vendor records store (D1). Process 2 pulls her sales history back out of D1, loads the current best-performing model from the model store (D2), generates the forecast, and logs which model was used. Process 3 takes that forecast, works out the reorder point, and decides whether a low-stock alert needs to go out, sending the result back to her over WhatsApp, with the alert logged back to D1. Nothing here happens outside the pipeline; the vendor's message is the only thing that ever comes in from outside.

3.3.4 Entity Relationship Diagram

SokoSmart — Entity Relationship Diagram

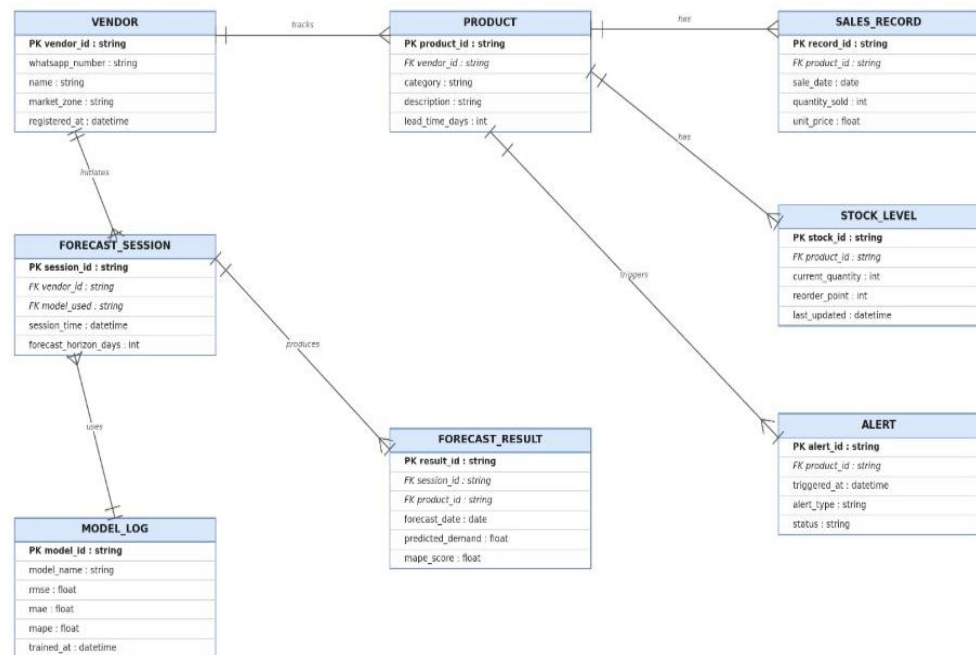


Figure 5 Entity relationship diagram

The Entity Relationship Diagram defines the eight core entities in SokoSmart's data layer and the relationships between them. The **VENDOR** entity is the central actor, identified by a unique vendor ID linked to their WhatsApp number and market zone. Each vendor tracks one or more **PRODUCT**s, each of which carries a lead time field used in reorder point calculations. Every product maintains **SALES_RECORD**s individual transaction entries recording quantity sold and date and a **STOCK_LEVEL** entry that holds the current quantity and the dynamically computed reorder point. When stock falls at or below the reorder point, an **ALERT** entity is created for that product, recording the trigger time and alert status. Vendors also initiate **FORECAST_SESSION**s, each linked to a specific **MODEL_LOG** entry recording which model was selected and its accuracy scores (RMSE, MAE, MAPE). Each session produces one or more **FORECAST_RESULT**s predicted demand figures per product per future date. The cardinality notations confirm that one vendor can track many products, one product can have many sales records and one stock level entry, and one forecast session can produce many forecast results.

3.3.5 Data Dictionary

Table X presents the data dictionary for SokoSmart's core entities, defining each field's data type, constraints, and purpose.

Table 5 Data Dictionary

Entity	Attribute	Data Type	Constraints	Description
VENDOR	vendor_id	string	PK, unique	Unique system identifier for each vendor
VENDOR	whatsapp_number	string	unique, not null	Vendor's registered WhatsApp number (E.164 format)
VENDOR	name	string	not null	Vendor's business or personal name
VENDOR	market_zone	string	not null	Market location : Gikomba, Eastleigh, Toi, or CBD
VENDOR	registered_at	datetime	auto	Timestamp of account creation
PRODUCT	product_id	string	PK, unique	Unique product identifier
PRODUCT	vendor_id	string	FK → VENDOR	Links product to owning vendor

PRODUCT	category	string	not null	Product category (e.g., dresses, shoes)
PRODUCT	description	string	nullable	Optional product details
PRODUCT	lead_time_days	int	> 0	Supplier delivery time in days
SALES_RECORD	record_id	string	PK, unique	Unique transaction record ID
SALES_RECORD	product_id	string	FK → PRODUCT	Links sale to product
SALES_RECORD	sale_date	date	not null	Date of sale transaction
SALES_RECORD	quantity_sold	int	≥ 1	Units sold in transaction
SALES_RECORD	unit_price	float	≥ 0	Price per unit in KES
STOCK_LEVEL	stock_id	string	PK, unique	Unique stock record ID
STOCK_LEVEL	product_id	string	FK → PRODUCT	One-to-one with product
STOCK_LEVEL	current_quantity	int	≥ 0	Current available stock
STOCK_LEVEL	reorder_point	int	≥ 0	Dynamically computed

				threshold
STOCK_LEVEL	last_updated	datetime	auto	Last modification timestamp
ALERT	alert_id	string	PK, unique	Unique alert identifier
ALERT	product_id	string	FK → PRODUCT	Product triggering alert
ALERT	triggered_at	datetime	auto	Alert generation timestamp
ALERT	alert_type	string	not null	Alert classification (e.g., low_stock)
ALERT	status	string	default 'active'	Alert state: active, dismissed, resolved
FORECAST_SESSION	session_id	string	PK, unique	Unique forecast run ID
FORECAST_SESSION	vendor_id	string	FK → VENDOR	Requesting vendor
FORECAST_SESSION	model_used	string	FK → MODEL_LOG	Model selected for session
FORECAST_SESSION	session_time	datetime	auto	Forecast execution timestamp
FORECAST_SESSION	forecast_horizon_days	int	> 0	Days ahead

				predicted
FORECAST_RESULT	result_id	string	PK, unique	Unique result entry ID
FORECAST_RESULT	session_id	string	FK → FORECAST_SESSION	Parent forecast session
FORECAST_RESULT	product_id	string	FK → PRODUCT	Product forecasted
FORECAST_RESULT	forecast_date	date	not null	Specific date of prediction
FORECAST_RESULT	predicted_demand	float	≥ 0	Forecasted quantity
FORECAST_RESULT	mape_score	float	≥ 0	Model accuracy for this prediction
MODEL_LOG	model_id	string	PK, unique	Unique model identifier
MODEL_LOG	model_name	string	not null	Model type: ARIMA, XGBoost, LSTM
MODEL_LOG	rmse	float	≥ 0	Root Mean Square Error
MODEL_LOG	mae	float	≥ 0	Mean Absolute Error
MODEL_LOG	mape	float	≥ 0	Mean Absolute Percentage Error

MODEL_LOG	trained_at	datetime	auto	Model training timestamp
------------	-------------	----------	------	--------------------------

3.4 Technical Requirements

This section outlines the hardware, software, and network requirements necessary for both the development of SokoSmart and its deployment and use in the target environment.

Requirements are separated into two categories: development requirements, which apply to the researcher building the system, and end-user requirements, which apply to vendors accessing the system through WhatsApp.

3.4.1 Development Requirements

The following tools, languages, frameworks, and hardware are required to build, train, and deploy SokoSmart.

Hardware Requirements (Development)

Table 6 Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i5 / AMD Ryzen 5 (8th gen+)	Intel Core i7 / AMD Ryzen 7
RAM	8 GB	16 GB
Storage	256 GB SSD	512 GB SSD
Operating System	Windows 10 / Ubuntu 20.04 / macOS 11	Windows 11 / Ubuntu 22.04
Internet connection	Broadband (10 Mbps+)	Broadband (25 Mbps+)

The RAM requirement of 8 GB minimum is driven by the computational demands of training LSTM neural networks locally. For LSTM training specifically, 16 GB is strongly recommended to avoid memory overflow during model fitting on larger datasets. Alternatively, Google Colaboratory provides free cloud-based GPU access that eliminates local hardware constraints for model training (Bisong, 2019).

Programming Languages

Table 7 programming languages

Language	Version	Purpose
Python	3.10+	Core development language for ML pipeline, Flask backend, and data processing

HTML / CSS	HTML5 / CSS3	Basic frontend for any administrative interface or documentation pages
JSON	—	Data interchange format between Flask API and Twilio webhook

Python is selected as the sole core programming language for SokoSmart based on its dominance in the machine learning ecosystem, its extensive library support for time-series forecasting and deep learning, and its compatibility with the Flask web framework used for API development (Géron, 2022).

Frameworks and Libraries

Table 8 Frameworks and Libraries

Library / Framework	Version	Purpose
Flask	3.0+	Backend web framework for API routing and webhook handling
scikit-learn	1.4+	ARIMA preprocessing, model evaluation metrics (RMSE, MAE, MAPE)
statsmodels	0.14+	ARIMA and SARIMA model implementation
XGBoost	2.0+	XGBoost gradient boosting model implementation
TensorFlow / Keras	2.15+	LSTM neural network model implementation
Pandas	2.0+	Data loading, cleaning, and transformation
NumPy	1.26+	Numerical computation and array operations
Matplotlib / Seaborn	3.8+ / 0.13+	Data visualization for model evaluation and EDA
Twilio Python SDK	8.0+	WhatsApp message sending and receiving via Twilio API
python-dotenv	1.0+	Environment variable management for API keys and secrets
pytest	7.0+	Automated unit and integration testing

Development Tools

Table 9 Development Tools

Tool	Purpose
Jupyter Notebook / Google Colab	Interactive model development, training, and evaluation

Visual Studio Code	Primary code editor for Flask backend and chatbot development
Git / GitHub	Version control and code repository management
Postman	API endpoint testing before Twilio integration
ngrok	Local Flask server tunnelling for Twilio webhook testing during development
Mendeley Desktop	Reference management for project documentation
Microsoft Word	Project documentation authoring

Cloud Deployment Tools

Table 10 Cloud Deployment Tools

Tool	Purpose
Render / Railway	Cloud hosting for Flask application deployment
Twilio Console	WhatsApp sandbox configuration and message log monitoring
GitHub Actions (optional)	Continuous integration and automated deployment pipeline

3.4.1.A Security Architecture

Security is handled at three levels. First, transport: every message between Twilio and the Flask backend travels over HTTPS with TLS 1.3, and I validate Twilio's webhook signature on every request to block spoofing. Second, data: vendor records are encrypted at rest through AES-256, which Render and Railway provide by default on their managed databases. WhatsApp numbers are hashed with SHA-256 inside my system; the original number only lives in Twilio's sandbox. Third, application: I use SQLAlchemy ORM for all database queries, which parameterises inputs automatically and prevents injection. I also do not send any vendor data to third-party analytics. This setup meets Kenya's Data Protection Act 2019 standards for lawful processing and data minimisation (Republic of Kenya, 2019).

3.4.2 End-User Requirements

SokoSmart is explicitly designed to impose minimal requirements on the end user. The following specifications define the minimum environment required for a vendor to access and use the system.

Hardware Requirements (End User)

Table 11 Hardware requirements

Component	Minimum Requirement
Device	Android smartphone (Android 5.0+) or iPhone (iOS 12+)

RAM	2 GB
Storage	100 MB free space (for WhatsApp only)
Data connection	Mobile data (2G minimum; 3G or 4G recommended for response speed)
Screen	Any screen size capable of running WhatsApp

No laptop, desktop computer, POS system, barcode scanner, or any additional hardware is required. The vendor's existing smartphone running WhatsApp is the only hardware requirement (Communications Authority of Kenya, 2024).

Software Requirements (End User)

Table 12 Software Requirements

Software	Version	Notes
WhatsApp Messenger	2.21+	Available free on Google Play Store and Apple App Store
Internet browser	Any	Required only if vendor accesses any optional web-based help documentation

No Python installation, no app download beyond WhatsApp, and no technical configuration is required from the vendor. The system is entirely server-side all computation occurs on the cloud-hosted Flask backend.

Connectivity Requirements (End User)

SokoSmart's WhatsApp interface is designed to function on standard mobile data as available across Nairobi's market zones. The (Communications Authority of Kenya, 2024) confirms that 3G and 4G coverage is near universal in Nairobi County, meaning the connectivity requirement is met by the existing infrastructure in all four target market zones. WhatsApp's offline message queuing feature additionally ensures that alerts generated while a vendor's phone is temporarily offline are delivered upon reconnection, providing resilience in low connectivity moments.

3.5 Project Budget

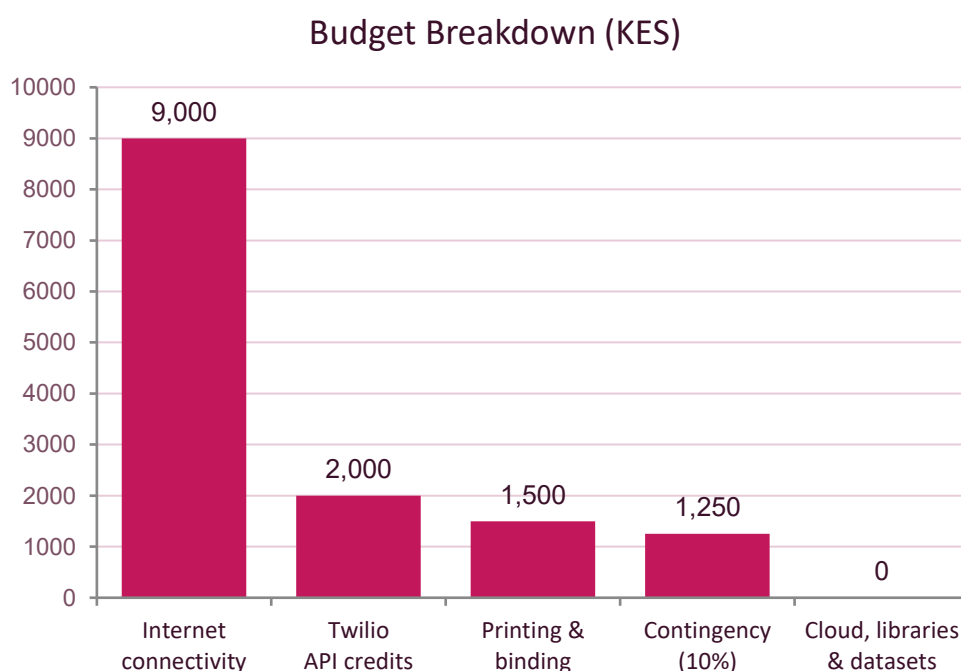
This section presents the estimated budget for developing and deploying SokoSmart. Costs are categorised into software and tools, cloud infrastructure, data acquisition, and miscellaneous expenses. All figures are quoted in Kenyan Shillings (KES) and reflect student pricing or free-tier availability where applicable.

Table 13 Budget

Item	Description	Unit Cost (KES)	Quantity	Total (KES)
Development laptop	Existing personal equipment (depreciation excluded)	0	1	0
Internet connectivity	Home broadband for development period	3,000/month	3 months	9,000
Twilio WhatsApp API	Sandbox usage free; testing credits for message volume	2,000	1	2,000
Cloud hosting (Render/Railway)	Free tier sufficient for prototype; paid tier contingency	0 (free tier)	1	0
Python libraries	Open source (scikit-learn, TensorFlow, Flask, etc.)	0	—	0
Datasets	Publicly available fashion retail datasets	0	—	0
Reference materials	Journal access via university library subscription	0	—	0
Printing and binding	Final project report	1,500	1	1,500

	production			
Contingency (10%)	Unforeseen expenses	—	—	1,250
Grand Total				**13,750**

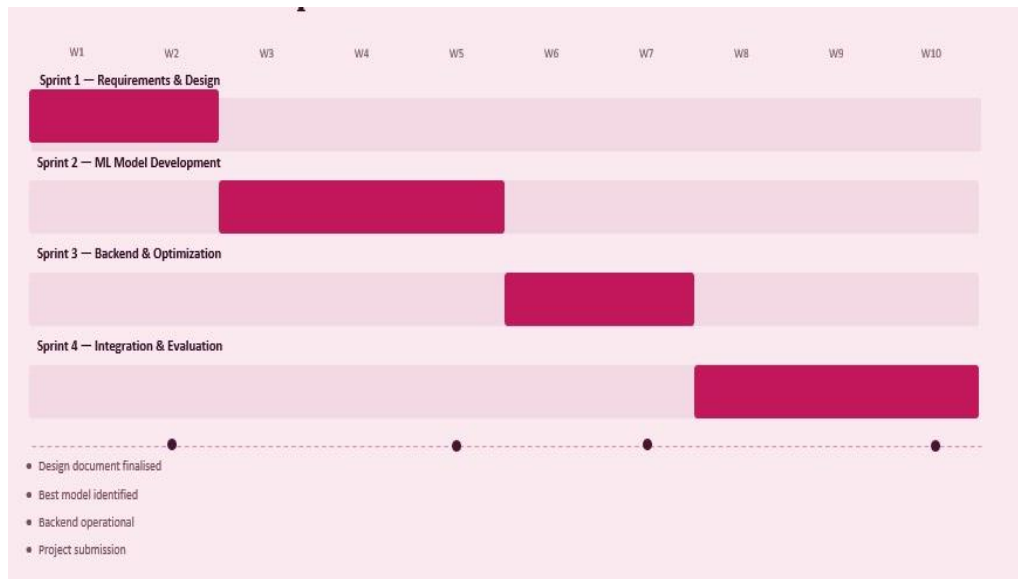
The budget totals KES 13,750, well within the affordability constraint established in NFR6 (KES 2,000 monthly operational cost). The largest single expense is internet connectivity, which is already a sunk cost for most students. Cloud hosting is absorbed through free-tier services, and all software is open-source. This cost structure confirms that SokoSmart is financially viable for both development and eventual deployment to micro-vendors who cannot absorb subscription fees.



3.6 PROJECT TIMELINE (GANTT CHART)

The project schedule spans 10 weeks, aligned with the Agile Scrum framework described in Section 3.1. Table X maps each sprint to specific deliverables, milestones, and dependencies.

Table 14 Gantt Chart SokoSmart Development Schedule



The schedule allows two weeks for requirements and design, three weeks for iterative model development (the longest phase due to training and tuning overhead), two weeks for backend construction, and three weeks for chatbot integration, testing, and final documentation. Week 5 includes a buffer for model selection delays, and Week 9 reserves time for usability testing with representative users. This timeline is realistic given the scope and aligns with the four-sprint Agile structure.

References

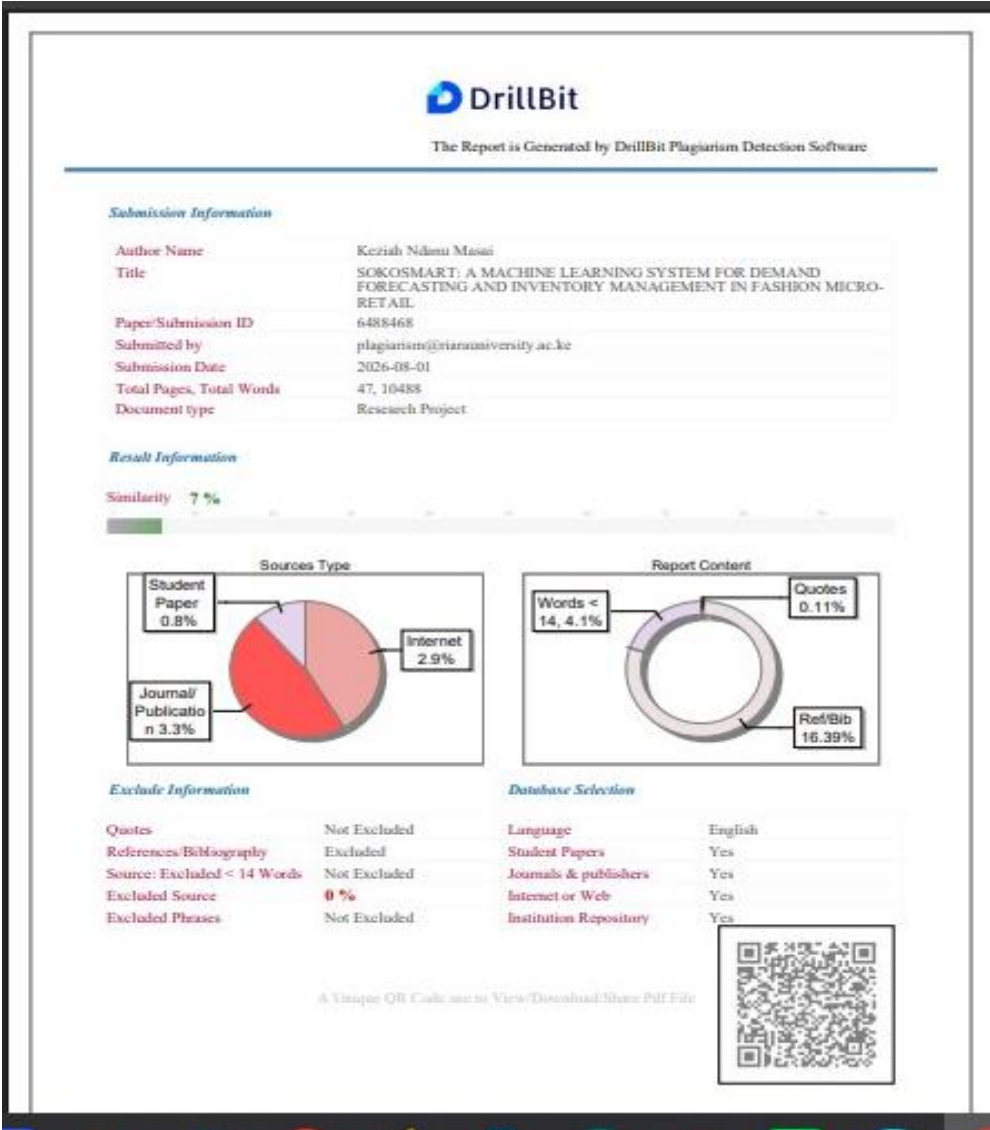
- Aker, J. C., & Mbiti, I. M. (2010). Mobile phones and economic development in Africa. *Journal of Economic Perspectives*, 24(3), 207–232.
<https://doi.org/10.1257/jep.24.3.207>
- Baryannis, G., Validi, S., Dani, S., & Antoniou, G. (2019). Supply chain risk management and artificial intelligence: State of the art and future research directions. *International Journal of Production Research*, 57(7), 2179–2202.
<https://doi.org/10.1080/00207543.2018.1530476>
- Bisong, E. (2019). *Building machine learning and deep learning models on Google Cloud Platform: A comprehensive guide for beginners*. Apress.
<https://doi.org/10.1007/978-1-4842-4470-8>
- Communications Authority of Kenya. (2024). *Quarterly sector statistics report: Quarter 1, 2023/2024*.
<https://www.ca.go.ke>
- Ferreira, K., Lee, B., & Simchi-Levi, D. (2016). Analytics for an online retailer: Demand forecasting and price optimization. *Manufacturing & Service Operations Management*, 18(1), 69–88.
<https://doi.org/10.1287/msom.2015.0561>
- Fildes, R., Ma, S., & Kolassa, S. (2022). Retail forecasting: Research and practice. *International Journal of Forecasting*, 38(4), 1283–1318.
<https://doi.org/10.1016/j.ijforecast.2019.06.004>
- Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
- Giri, C., Jain, S., Zeng, X., & Bruniaux, P. (2022). Deep learning for demand forecasting in the fashion and apparel retail industry. *Forecasting*, 4(2), 565–581.
<https://doi.org/10.3390/forecast4020031>
- Hollupathirage, I. (2025). *A digital twin system for retail inventory and sales management* [Master's thesis, University of Vaasa]. <https://urn.fi/URN:NBN:fi-fe20251128112774>
- IHL Group. (2023). *Retail's \$1.75 trillion problem: The impact of out-of-stocks and overstocks*.
<https://www.ihlservices.com>
- Jack, W., & Suri, T. (2014). Risk sharing and transactions costs: Evidence from Kenya's mobile money revolution. *American Economic Review*, 104(1), 183–223.
<https://doi.org/10.1257/aer.104.1.183>

- Kharfan, M., Chan, V. W. K., & Okumus, F. (2020). A data-driven forecasting approach for newly launched seasonal products by leveraging machine-learning approaches. *Annals of Operations Research*, 303, 159–187. <https://doi.org/10.1007/s10479-020-03666-w>
- Loureiro, A. L., Miguéis, V. L., & Da Silva, L. F. (2018). Exploring the use of deep neural networks for sales forecasting in fashion retail. *Decision Support Systems*, 114, 81–93. <https://doi.org/10.1016/j.dss.2018.08.010>
- Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018). Statistical and machine learning forecasting methods: Concerns and ways forward. *PLOS ONE*, 13(3), e0194889. <https://doi.org/10.1371/journal.pone.0194889>
- Omwenga, E., Waweru, M., & Kamau, J. (2020). Competitive dynamics and survival strategies among micro-retailers in the Nairobi Central Business District. *African Journal of Business and Economic Research*, 15(2), 112–130. <https://doi.org/10.31920/1750-4562/2020/v15n2a6>
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.
- Ren, S., Chan, H. L., & Siqin, T. (2020). Demand forecasting in retail operations for fashionable products: Methods, practices, and real case study. *Annals of Operations Research*, 291, 761–777. <https://doi.org/10.1007/s10479-019-03148-8>
- Republic of Kenya. (2019). *Data Protection Act, 2019*. https://kenyalaw.org/kl/fileadmin/pdfdownloads/Acts/2019/TheDataProtectionAct__No24of2019.pdf
- Selamat, M. A. A., & Windasari, N. A. (2021). Chatbot for SMEs: Integrating customer and business owner perspectives. *Technology in Society*, 66, 101685. <https://doi.org/10.1016/j.techsoc.2021.101685>
- Seyedan, M., Mafakheri, F., & Wang, C. (2023). Order-up-to-level inventory optimization model using time-series demand forecasting with ensemble deep learning. *Supply Chain Analytics*, 3, 100024. <https://doi.org/10.1016/j.sca.2023.100024>
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson Education.
- Swaminathan, K., Wang, X., Goh, M., Li, J., Zhang, S., Liu, Y., Zheng, S., & Chen, S. (2023). Demand forecasting for fashion products: A systematic review. *International Journal of Forecasting*.

<https://doi.org/10.1016/j.ijforecast.2022.08.001>

Wanjiru, R., & Ngugi, B. (2021). Inventory management practices and financial performance of small and medium enterprises in Nairobi County, Kenya. *Journal of Business and Management*, 23(1), 45–58.
<https://doi.org/10.9790/487X-2301024558>

APPENDIX A: PLAGARISM REPORT



DrillBit Similarity Report

7

SIMILARITY %

53

MATCHED SOURCES

A

GRADE

A-Satisfactory (0-10%)

B-Upgrade (11-40%)

C-Poor (41-60%)

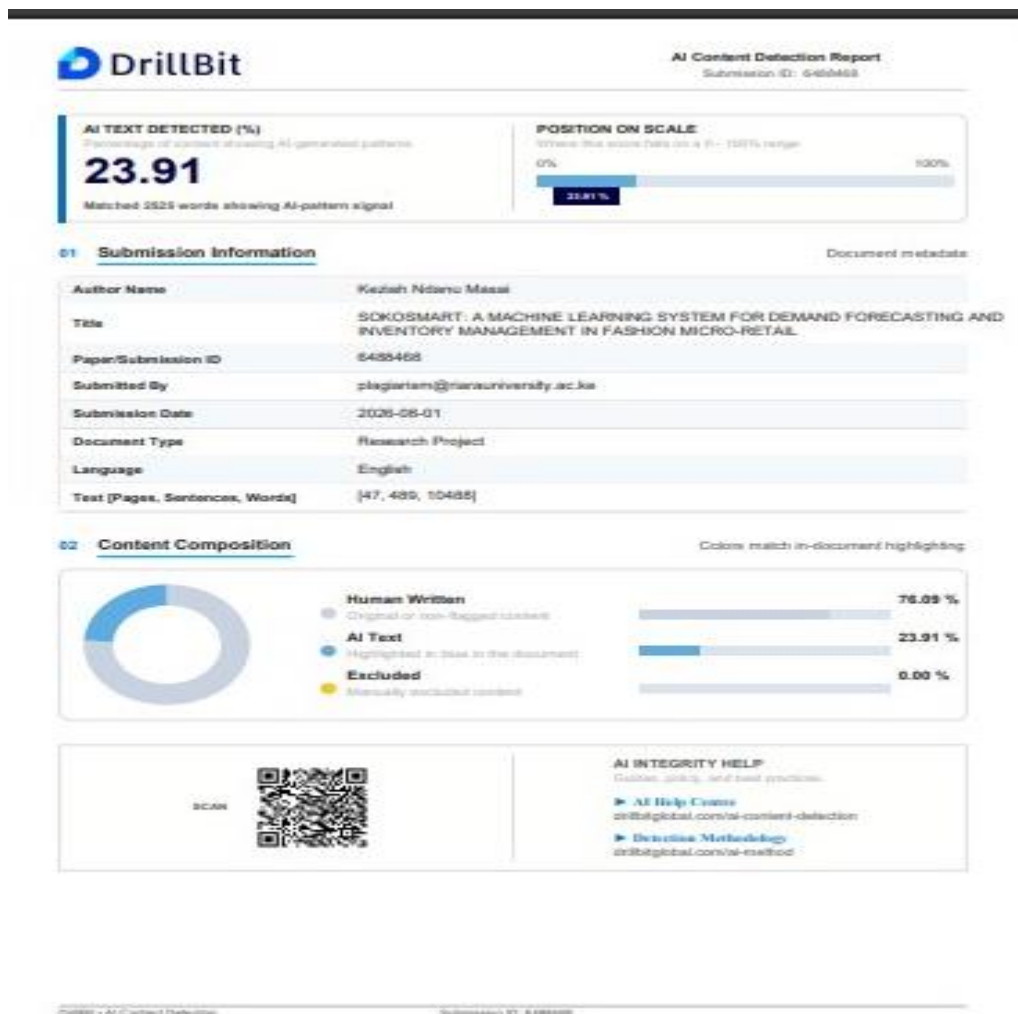
D-Unacceptable (61-100%)

LOCATION	MATCHED DOMAIN	%	SOURCE TYPE
3	vdoc.pub	<1	Internet Data
4	www.bits-pilani.ac.in	<1	Publication
5	www.scribd.com	<1	Internet Data
6	REPOSITORY - Submitted to Exam section VTU on 2026-02-10 11-17 4376297	<1	Student Paper
7	REPOSITORY - Submitted to Exam section VTU on 2026-02-10 11-50 4376844	<1	Student Paper
8	www.serianu.com	<1	Publication
10	qjcsenline.org	<1	Publication
11	REPOSITORY - Submitted to Exam section VTU on 2026-02-06 12-43 4422692	<1	Student Paper
12	pdfcoffee.com	<1	Internet Data
13	pdfcoffee.com	<1	Internet Data
15	medium.com	<1	Internet Data
16	moam.info	<1	Internet Data
18	isteonline.org	<1	Publication

19	dl.lib.uom.lk	<1	Publication
20	gredos.usal.es	<1	Publication
21	iris.cnr.it	<1	Publication
22	udsspace.uds.edu.gh	<1	Publication
23	escholarship.org	<1	Internet Data
24	REPOSITORY - Submitted to Exam section VTU on 2026-02-11 10-57 4393874	<1	Student Paper
26	epublications.marquette.edu	<1	Publication
27	krishikosh.egranth.ac.in	<1	Internet Data
28	Thesis Submitted to Shodhganga Repository	<1	Publication
29	www.freepatentsonline.com	<1	Internet Data
31	vlinkinfo.com	<1	Internet Data
32	AN ANALYSIS OF FORECASTING METHODS USING SAME DAY OF THE WEEK VERSUS SAME DAY by LE-2004	<1	Publication
34	forum.effectivealtruism.org	<1	Internet Data
35	homeguides.sfgate.com	<1	Internet Data
36	kyuspace.kyu.ac.ug	<1	Publication
37	Manipulating the context The use of English borrowings as a discourse strategy by CANAGARAJAH-1995	<1	Publication
38	opus.govst.edu	<1	Publication
39	Thesis submitted to shodhganga - shodhganga.inflibnet.ac.in	<1	Publication

40	vdoc.pub	<1	Internet Data
42	www.thefreelibrary.com	<1	Internet Data
43	IEEE 2013 Humaine Association Conference on Affective Computing and by	<1	Publication
45	llibrary.co	<1	Internet Data
46	Acoustic emission monitoring of an eighteenth-century wardrobe to support a stra by Strojecki-2014	<1	Publication
47	acropolium.com	<1	Internet Data
48	Blockchain based decentralized local energy flexibility market, by Antal, Claudia, Yr-2021	<1	Publication
49	docplayer.info	<1	Internet Data
50	etheses.uin-malang.ac.id	<1	Publication
52	ijarccce.com	<1	Publication
53	index-of.es	<1	Publication
54	ir-library.ku.ac.ke	<1	Publication
55	krishikosh.egranth.ac.in	<1	Internet Data
56	slma.lk	<1	Publication
57	storre.stir.ac.uk	<1	Publication
58	unstats.un.org	<1	Publication
59	vdoc.pub	<1	Internet Data
60	www.bits-pilani.ac.in	<1	Publication

61	www.gradesgroom.com	<1	Internet Data
62	www.openphilanthropy.org	<1	Internet Data
63	www.readbag.com	<1	Internet Data
64	www.scribd.com	<1	Internet Data



APPENDIX B: STRUCTURED OBSERVATION NOTES

A.1 Observation Protocol and Method

I observed eight fashion vendors in four Nairobi markets: Gikomba, Eastleigh, Toi Market, and Nairobi CBD. Two vendors were observed in each market between 15 and 20 July 2026. I looked at how they kept track of stock, when they decided to buy more stock, what happened when items were left unsold or ran out, and how they used their phones in the business. I did not record vendor names, stall numbers, photographs, or personal information.

A.2 Observation Notes

Gikomba Market

Vendor G1; Mitumba clothing

- Used a small notebook to keep track of stock.
- When jeans started running low, the vendor judged the amount left rather than doing an actual count.
- Three unopened bales of children's sweaters were behind the stall. The vendor said they had been bought the previous month when the weather was cold but were no longer selling well.
- A customer asked for size 34 men's jeans, but the vendor did not have any and directed the customer to another stall.
- The vendor had a Tecno phone and was using WhatsApp to communicate with a supplier.

Vendor G2; Women's dresses and tops

- Did not have a visible stock record. The vendor counted the remaining black dresses aloud when asked.
- She said she normally restocked on Saturdays based on what looked low.
- Some floral blouses were still on the rack even though they had been marked down from KES 800 to KES 300. The vendor said they were from December and the trend had changed.
- She used WhatsApp Business to show customers product photos, but the catalogue was not connected to her stock records.

Eastleigh Market

Vendor E1; Men's shirts and trousers

- Used the Notes app on a Samsung phone to keep a list of some stock.
- The list had quantities for different shirts and trousers but had no dates. The vendor said it had not been updated for about two weeks.
- A customer asked for size 34 trousers, but none were available. The vendor immediately sent a WhatsApp voice note to a supplier asking for four pairs.
- A carton of leather belts bought during Ramadan was still unopened.
- The vendor was comfortable using WhatsApp for supplier communication and customer messages.

Vendor E2; Women's hijabs and modest wear

- Shared a notebook with her sister, who operated another stall nearby.
- Some stock figures in the notebook did not match the physical stock, and several entries had been crossed out.
- A customer asked for navy-blue school hijabs, but the vendor said they had finished the previous day.
- The vendor estimated that the shortage had cost about KES 8,000 during the back-to-school period.
- WhatsApp was used for supplier orders and customer inquiries.

Toi Market

Vendor T1; Children's clothing

- Did not use a notebook or phone application to record stock.
- When asked how she decided what to buy, she said, "**Najua biashara yangu**" (I know my business).
- The 0–3 month baby rompers were still in good supply, while the 6–9 month section was almost empty.
- The vendor said she would go to Gikomba to buy more stock when she had enough money.
- Some of the 0–3 month rompers had dust on their packaging. She said customers were now buying bigger sizes and she had not noticed the change early enough.
- She used a basic Tecno phone and mainly used WhatsApp for receiving messages.

Vendor T2; Shoes

- Used a school exercise book with tally marks to record sales.
- The pages were divided by shoe type, but there were no dates.
- He restocked popular sizes when the tally became low but sometimes continued ordering slow-moving sizes.
- Several pairs of large men's formal shoes had remained on a high shelf and were covered with dust.
- He received supplier photos through WhatsApp and preferred voice notes to typing.

Nairobi CBD

Vendor C1; Office wear and formal shirts

- Used a spreadsheet on a phone, but the spreadsheet was maintained by a nephew who visited on weekends.
- The vendor sent stock changes to the nephew through WhatsApp.
- Some blazers bought for the December period were still in stock and had been reduced from KES 3,500 to KES 1,500.
- White shirts in size 16.5 regularly ran out, but the order quantity was not increased.
- The vendor was comfortable using WhatsApp.

Vendor C2; Casual wear

- Did not use a visible stock record and relied mainly on experience.
- The vendor normally travelled to River Road suppliers on Monday mornings with money saved from the weekend.
- Some graphic T-shirts had become difficult to sell after the trend changed.
- The vendor said jeans in sizes 32 and 34 regularly ran out around the end of the month.
- WhatsApp was mainly used for family communication, and the vendor was doubtful about using business applications.

A.3 Main Findings from the Observations

The observations showed that the vendors used different methods to keep track of stock, but none of the eight vendors was using a dedicated inventory management system. Some relied on notebooks or exercise books, while others used phone notes, spreadsheets maintained by someone else, or memory.

Stock was usually reordered after it became low or when a customer asked for an item that was unavailable. There were also examples of products remaining unsold after trends or seasonal demand had changed. At the same time, some vendors regularly ran out of particular sizes during periods of higher demand.

WhatsApp was already familiar to the vendors, mainly for communicating with customers and suppliers. However, their comfort with typing and other phone-based tasks differed. These observations helped me identify the need for a simple inventory system that could provide demand forecasts and reorder alerts through a platform the vendors already use.

